



INSTITUTO DE FORMACIÓN TÉCNICA SUPERIOR N.º 18

Tecnicatura Superior en Ciencia de Datos e Inteligencia Artificial

SEMINARIO DE ACTUALIZACIÓN

Clase 2

Preparando nuestro entorno de desarrollo

Python · Visual Studio Code · Entornos virtuales · Git

```
proyecto_integrador  
  
> py -m venv .venv  
> .venv\Scripts\activate  
(.venv) > git init  
(.venv) > git commit -m "..."
```

DOCENTE

Cynthia M. Villagra

CICLO LECTIVO

2026 · Segundo cuatrimestre

FECHA

21 de agosto de 2026

¿Hacia dónde vamos?

El recorrido completo del seminario. Hoy trabajamos sobre los dos primeros escalones.



PREGUNTA ACTIVADORA

Si al final del seminario queremos tener una aplicación publicada en internet, ¿qué necesitamos preparar primero en nuestra computadora?

¿Qué es el Proyecto Integrador?

Todo lo que hagamos de acá en adelante se suma a la misma carpeta que creamos hoy.

QUÉ ES

Una aplicación web propia

Con interfaz, que usa un modelo de inteligencia artificial y queda publicada en internet.

QUÉ ENTREGAMOS

Repositorio + app funcionando

El repositorio en GitHub, la aplicación en Hugging Face Spaces y su README.

Y EL TEMA

Lo elegís vos

No hace falta decidirlo hoy. En la Clase 3, con la interfaz andando, se ve mucho mejor el abanico.

El proyecto se construye por capas: cada clase agrega una sobre la anterior. Hoy ponemos los cimientos.

Al terminar la clase de hoy vas a poder...

- verificar qué intérprete de Python ejecuta tu código
- explicar por qué un proyecto necesita un entorno aislado
- fundamentar qué se versiona y qué no en un proyecto Python
- distinguir python, py y el intérprete que usa VS Code
- crear y activar un entorno virtual e instalar una dependencia
- inicializar un repositorio Git y hacer tu primer commit

Estos seis puntos vuelven al final de la clase como checklist. Si alguno queda flojo, avisá: lo retomamos.

Si algo no te funciona

Instalar herramientas es difícil para todo el mundo. También para mí. No es un problema tuyo.

PASO 1

Pegalo en el chat

Copía el mensaje de error completo. Los errores del grupo son el material de esta clase.

PASO 2

Duplas de apoyo

Quien ya tiene todo andando acompaña a quien está trabado. Se aprende igual mirando y preguntando.

PASO 3

Plan B

Si tu máquina no te deja instalar: Google Colab o GitHub Codespaces, sin instalar nada.

Todo queda grabado y tenés el **machete de comandos** descargable en el aula virtual. La actividad de hoy no lleva nota: podés completarla después.

Antes que nada: ¿qué tenés instalado?

Ejecutá los comandos de tu sistema operativo y pegá el resultado en el chat. Todavía no arreglamos nada: solo miramos.

Windows — PowerShell

```
> python --version  
> py --version  
> py -0  
> code --version  
> git --version
```

macOS / Linux — Terminal

```
$ python3 --version  
$ code --version  
$ git --version
```

Es muy probable que no a todos les responda lo mismo. Eso es exactamente el tema de los próximos 25 minutos.

«Instalé Python... pero no sé qué Python estoy usando.»

Python 3.11

del año pasado

Python 3.13

de anteayer

Python 3.9

que vino con algo

PREGUNTA ACTIVADORA

*¿Podemos tener más de una versión de Python instalada en la misma computadora?
¿Y cómo sabemos cuál se ejecuta cuando escribimos python?*

Tener Python instalado no es saber cuál se ejecuta

COMANDO

python

Ejecuta el intérprete asociado al comando del sistema. Uno solo, el que quedó primero.

LANZADOR (SOLO WINDOWS)

py

Es el Python Launcher: puede listar todas las versiones instaladas y elegir una.

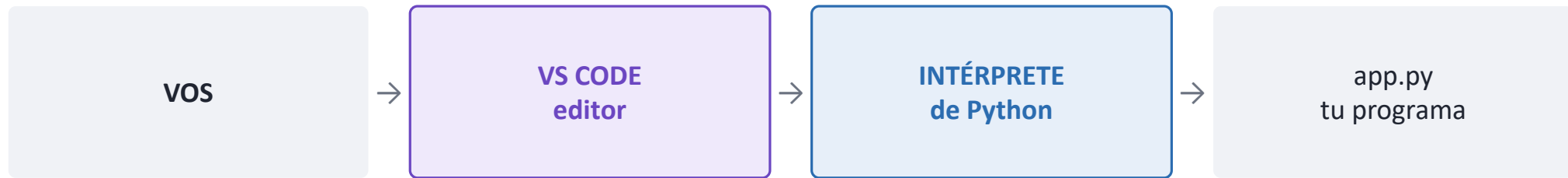
```
Windows — PowerShell

> py -0
-V:3.13 *      Python 3.13 (64-bit)
-V:3.11       Python 3.11 (64-bit)

> py -3.11     # abrimos una versión concreta
```


PROBLEMA

«Tengo Python instalado y abrí mi código en VS Code. ¿VS Code es Python?»



Escribir el código y ejecutarlo son dos cosas distintas, y las hacen dos programas distintos.

PREGUNTA ACTIVADORA

Si tenemos varias versiones instaladas, ¿cómo sabe VS Code cuál tiene que usar?

Git: instalar y presentarse

Git necesita saber quién sos para poder firmar cada versión que guardes. Se configura una sola vez en la vida.

Windows / macOS / Linux — es igual en los tres

```
> git --version

> git config --global user.name "Nombre Apellido"
> git config --global user.email "tu@correo.com"
> git config --global init.defaultBranch main

> git config --global --list    # verificamos
```

OJO

Sin esto, no hay commit

Si te salteás estas dos líneas, más tarde Git te va a responder «Please tell me who you are» y no vas a poder guardar nada.

Tres herramientas, tres trabajos distintos

PYTHON

Ejecuta

Es el intérprete: toma tu archivo .py y lo corre.

VISUAL STUDIO CODE

Editamos

Es el lugar donde escribimos y organizamos el proyecto.

GIT

Registra

Guarda el historial de versiones del proyecto, en tu computadora.

Git y GitHub no son lo mismo. Hoy trabajamos solo con Git, y todo queda en tu máquina.

Palabras en inglés que vas a ver hoy

Las herramientas están en inglés y los mensajes de error también. No hace falta saber inglés: hace falta reconocer estas doce palabras.

interpreter → intérprete

virtual environment → entorno virtual

repository → repositorio

untracked → sin seguimiento

branch → rama

is not recognized → no se reconoce el comando

environment → entorno

dependency → dependencia

commit → versión guardada

staged → preparado para guardar

freeze → congelar la lista

command not found → no encuentro ese comando

Dos proyectos, la misma librería, distinta versión

PROYECTO A

Librería X 1.0

Python 3.11 · funciona perfecto desde el año pasado.

VS

PROYECTO B

Librería X 2.0

Python 3.11 · lo empezaste esta semana y necesita la versión nueva.

PREGUNTA ACTIVADORA

*¿Qué versión de la librería instalamos?
¿Qué puede pasar si todos los proyectos usan las librerías instaladas globalmente?*

Hagámoslo peor

Sumamos un tercer proyecto. Y este además necesita otra versión de Python.

PROYECTO A

Python 3.11 · X 1.0

PROYECTO B

Python 3.11 · X 2.0

PROYECTO C

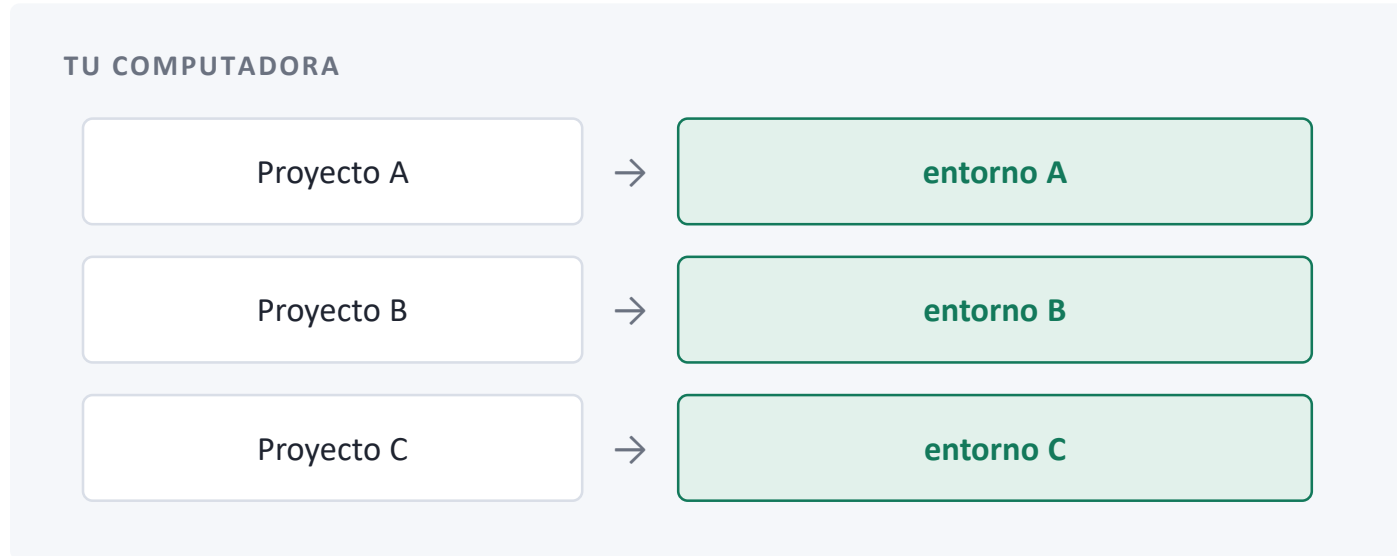
Python 3.13 · X 1.4 + otras 12

Todo instalado en el mismo lugar

¿Qué problemas trae? Anotemos los que salgan.

Recuperar del grupo: conflictos, incompatibilidades, dificultad para reproducir el proyecto en otra máquina.

La solución: un entorno para cada proyecto



Aislamiento

Lo que instalás en un proyecto no toca a los demás.

Organización

Cada proyecto declara qué necesita para funcionar.

Reproducibilidad

Otra persona puede rearmar tu entorno igual que el tuyo.

¿Hay una sola herramienta para hacerlo?

No. Hay varias, y todas resuelven el mismo problema con distinto alcance.

LA DEL SEMINARIO

venv + pip

Viene con Python, no hay que instalar nada y deja ver el concepto con claridad.

MUY USADA EN DATOS

Conda

Administra entornos y paquetes, y además puede manejar la versión de Python.

FLUJOS MODERNOS

uv

Entornos, dependencias y proyectos, mucho más rápido. No la instalamos hoy.

Empezamos con venv + pip. La comparación completa de las tres quedó como lectura en el aula virtual.

Laboratorio 1: armamos el proyecto

- 1 Creá la carpeta proyecto_integrador y abrila en VS Code
- 2 Creá dentro dos archivos: app.py y README.md
- 3 Antes de nada, ejecutá la línea de la derecha y guardá el resultado
- 4 Creá el entorno virtual y activalo
- 5 Volvé a ejecutar exactamente la misma línea

terminal integrada de VS Code — Ctrl + ñ

la línea del paso 3 y del paso 5

```
> python -c "import sys;  
        print(sys.executable)"
```

crear y activar (Windows)

```
> py -m venv .venv  
> .venv\Scripts\activate
```

macOS / Linux

```
$ python3 -m venv .venv  
$ source .venv/bin/activate
```

Cuando el entorno está activo, la terminal muestra (.venv) al principio de la línea.

OBSERVAMOS

La misma orden, dos respuestas distintas

● ● ● ANTES de crear el entorno

```
> python -c "import sys;  
    print(sys.executable)"
```

```
C:\Users\vos\AppData\...  
    \Python313\python.exe
```

● ● ● DESPUÉS de activarlo

```
(.venv) > python -c "import sys;  
    print(sys.executable)"
```

```
C:\...\proyecto_integrador  
    \.venv\Scripts\python.exe
```

PREGUNTA ACTIVADORA

*El comando es idéntico y la respuesta cambió. ¿Qué pasó?
¿Dónde se van a instalar ahora las librerías que agreguemos?*

Pegá tus dos rutas en el chat: la de antes y la de después.

Pausa

10 minutos

Si algo no te funcionó, quedate: lo resolvemos ahora, con calma y sin apuro.

Si ya tenés todo andando, nos vemos a las 21:30.

 en pausa

```
> pausa --min 10  
volvemos 21:30
```

Tres acciones distintas que se confunden

Podés tener una sin las otras dos, y ahí empiezan los problemas raros.

PASO 1

Crear

Aparece la carpeta `.venv` en el proyecto.

```
py -m venv .venv
```

PASO 2

Activar

La terminal empieza a usar ese Python. Se nota en el `(.venv)`.

```
.venv\Scripts\activate
```

PASO 3

Seleccionar

VS Code usa ese Python cuando apretás Run.

Python: Select Interpreter

● ● ● app.py

```
# lo comprobamos desde el propio programa
```

```
> import sys ; print(sys.executable) ; print(sys.version)
```

Tu proyecto funciona. ¿Cómo se lo pasás a alguien?

≈ 3.000

archivos dentro de tu carpeta .venv

y son distintos en cada computadora

PREGUNTA ACTIVADORA

¿Le mando todo eso?

¿O hay alguna forma de decirle «necesitás estas tres librerías» y que se las instale solo?

Dos cosas parecidas que hacen lo contrario

LA CARPETA

.venv

El entorno físico de TU computadora. Pesado, reconstruible y distinto en cada máquina. NO se versiona.

EL ARCHIVO

requirements.txt

La descripción de las dependencias. Tres líneas de texto que permiten rearmar el entorno. Sí se versiona.

terminal — proyecto_integrador

```
(.venv) > pip install requests
```

```
(.venv) > pip freeze > requirements.txt      # guardamos la lista
```

```
(.venv) > pip install -r requirements.txt    # y cualquiera la reconstruye
```

PROBLEMA

¿Alguna vez organizaste versiones así?

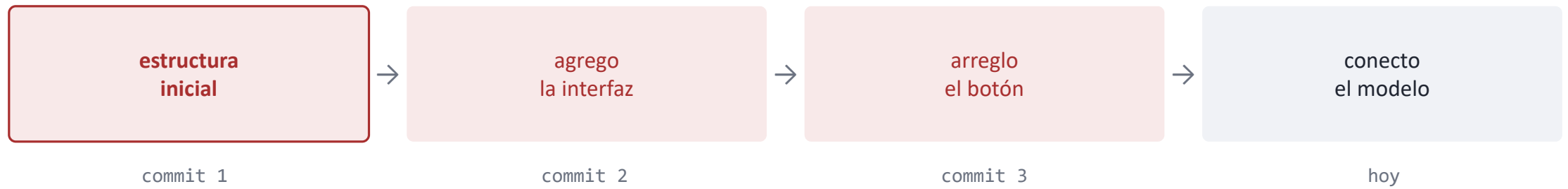
```
proyecto.py  
proyecto_final.py  
proyecto_final2.py  
proyecto_final_bueno.py  
proyecto_final_AHORA_SI.py
```

PREGUNTA ACTIVADORA

¿Qué problemas trae guardar versiones de esta manera? Dentro de tres meses, ¿cuál era la buena?

Git guarda el historial por vos

Cada commit es una foto del proyecto con un mensaje que explica qué cambió.



```
terminal — proyecto_integrador

> git init
> git status -u      # ¿qué ve Git en la carpeta?
```

La opción -u le pide a Git que muestre todos los archivos, uno por uno, sin agrupar carpetas.

Esto es lo que Git se está por llevar

terminal

```
> git status -u
```

Untracked files:

```
.venv/Scripts/python.exe  
.venv/Lib/site-packages/...  
.venv/Lib/site-packages/...  
... y 2.994 archivos más  
app.py  
README.md
```

¿Tiene sentido guardar en el historial una carpeta que podemos reconstruir en 20 segundos?

Le decimos a Git qué ignorar:

.gitignore

```
.venv/  
__pycache__/
```

Primero el .gitignore, después el primer commit. En ese orden.

PREGUNTA TRAMPA

¿Ya estamos usando GitHub?

Y si GitHub desapareciera esta noche, ¿tu repositorio seguiría funcionando?

Git y GitHub no son lo mismo

LO DE HOY

Git

Un programa instalado en tu computadora que lleva el control de versiones de tu proyecto. Funciona sin internet.

LA PRÓXIMA CLASE

GitHub

Un servicio en internet que aloja repositorios y permite compartirlos y colaborar. Usa Git, pero no es Git.

Hoy todo lo que hiciste vive en tu máquina. En la Clase 3 lo subimos.

CIERRE

¿Qué construimos hoy?

- ✓ Python instalado y verificado
- ✓ VS Code con la extensión Python
- ✓ Carpeta del Proyecto Integrador
- ✓ requirements.txt generado
- ✓ Git configurado
- ✓ Sabés qué intérprete ejecuta tu código
- ✓ Intérprete del entorno seleccionado
- ✓ Entorno virtual creado y activado
- ✓ .gitignore con .env y __pycache__
- ✓ Repositorio local con tu primer commit

Lo que sigue

GitHub



Gradio



Hugging Face



App publicada

¿Qué le falta a nuestro código para que una persona pueda usarlo sin ver una sola línea de Python?



Antes de irte

Se responde en el foro del aula virtual. Queda abierto 48 horas y no lleva nota.

1 · Pegá el resultado de estos dos comandos

● ● ● copió la salida tal cual

```
> python -c "import sys;
    print(sys.executable)"
> git log --oneline
```

2 · Elegí una pregunta y respondela

- ¿Para qué sirve un entorno virtual?
- ¿Qué diferencia hay entre .venv y requirements.txt?
- ¿Qué diferencia hay entre Git y GitHub?
- Si VS Code ejecuta otro Python, ¿qué revisarías?

Respondé como te resulte más cómodo: por escrito, con una captura comentada o con un audio de un minuto.